K.R. Mangalam University - School of Engineering & Technology

Task 1: Supervised ML for Intrusion Detection

Dataset: NSL-KDD (KDDTrain+) | Models: Decision Tree, Random Forest, SVM

Name	Yash Raj	Roll No. / Section	2401010008 / D
Programme	B.Tech Computer Science & Engineering	Subject	AI in Cyber Security

1. Objective

Build and evaluate supervised machine-learning classifiers that separate normal network connections from attacks, using the NSL-KDD dataset. The tasks are: load and explore the dataset (features and class distribution), apply at least three classifiers (Decision Tree, Random Forest, SVM), split the data 80% / 20% and apply label encoding where required.

2. Dataset Description

NSL-KDD is the cleaned version of the KDD Cup 1999 intrusion-detection data (duplicate records removed). The **KDDTrain+** file was used: 125,973 connection records with 41 features, an attack-name label and a difficulty score (dropped). No real network was accessed.

Property	Value
Records	125,973 (0 missing values, 0 duplicate rows)
Features	41 = 38 numeric + 3 categorical (protocol_type, service, flag)
Feature groups	Basic (duration, protocol, service, bytes), content (failed logins, root shell, ...), time-based traffic (count, serror_rate, ...), host-based traffic (dst_host_*)
Binary target	Normal = 67,343 (53.46%) | Attack = 58,630 (46.54%)
Attack categories	DoS 45,927 | Probe 11,656 | R2L 995 | U2R 52 (Normal 67,343)
Most common attacks	neptune 41,214 | satan 3,633 | ipsweep 3,599 | portsweep 2,931 | smurf 2,646

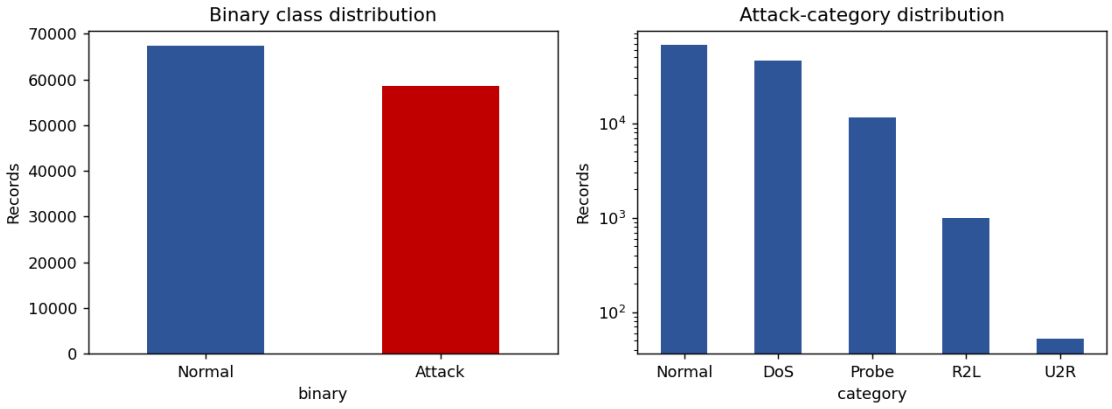


Fig 1: Class distribution (binary and attack category, log scale). The binary problem is almost balanced; R2L and U2R are very rare.

3. Method

- **Binary task:** Normal vs Attack (all attack types merged), so every model solves the same problem.
- **Label encoding:** protocol_type, service and flag were converted to integers with LabelEncoder; the target was encoded as Attack = 0, Normal = 1.
- **Split:** 80% training (100,778 rows) / 20% testing (25,195 rows), stratified, random_state = 42.
- **Models:** Decision Tree (default depth), Random Forest (100 trees), SVM with RBF kernel (features standardised; trained on a random 20,000-row subset of the training set because kernel SVM scales

poorly with rows - it is still tested on all 25,195 test rows).

- **Metrics:** accuracy, precision, recall, F1 (Attack = positive class) and the confusion matrix.

4. Source Code

```
import pandas as pd, numpy as np, time
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
    ConfusionMatrixDisplay

cols = ['duration','protocol_type','service','flag','src_bytes','dst_bytes','land','wrong_fragment','urgent','hot',
 'num_failed_logins','logged_in','num_compromised','root_shell','su_attempted','num_root','num_file_creations',
 'num_shells','num_access_files','num_outbound_cmds','is_host_login','is_guest_login','count','srv_count',
 'serror_rate','srv_serror_rate','rerror_rate','srv_rerror_rate','same_srv_rate','diff_srv_rate','srv_diff_host_rate',
'dst_host_count','dst_host_srv_count','dst_host_same_srv_rate','dst_host_diff_srv_rate','dst_host_same_src_port_rate',
 'dst_host_srv_diff_host_rate','dst_host_serror_rate','dst_host_srv_serror_rate','dst_host_rerror_rate',
 'dst_host_srv_rerror_rate','attack','difficulty']
df = pd.read_csv('KDDTrain+.txt', names=cols).drop(columns='difficulty')
print('Shape:', df.shape)
print('Missing values:', int(df.isna().sum().sum()), '| Duplicate rows:', int(df.duplicated().sum()))
print('Categorical features:', ['protocol_type','service','flag'])
print('Numeric features:', len(cols)-1-1-3+0, '(+3 categorical, +1 label)')
print(df.dtypes.value_counts().to_string())
print('\nProtocol types:'); print(df.protocol_type.value_counts().to_string())
print('\nUnique services:', df.service.nunique(), '| unique flags:', df.flag.nunique())
print('\nAttack-name distribution (top 10):'); print(df.attack.value_counts().head(10).to_string())
df['binary'] = np.where(df.attack=='normal','Normal','Attack')
cat_map = {'normal':'Normal',
 **{a:'DoS' for a in
    ['neptune','back','land','pod','smurf','teardrop','apache2','udpstorm','processtable','worm','mailbomb']},
 **{a:'Probe' for a in ['satan','ipsweep','nmap','portsweep','mscan','saint']},
 **{a:'R2L' for a in ['guess_passwd','ftp_write','imap','phf','multihop','warezmaster','warezclient','spy','xlock','xs
    noop','snmpguess','snmpgetattack','httptunnel','sendmail','named']},
 **{a:'U2R' for a in ['buffer_overflow','loadmodule','rootkit','perl','sqlattack','xterm','ps']}}
df['category'] = df.attack.map(cat_map)
print('\nAttack-category distribution:'); print(df.category.value_counts().to_string())
print('\nBinary class distribution:'); print(df.binary.value_counts().to_string());
    print((df.binary.value_counts(normalize=True)*100).round(2).to_string())

fig,ax=plt.subplots(1,2,figsize=(10,3.8))
df.binary.value_counts().plot.bar(ax=ax[0],color=['#2E5597','#C00000'],rot=0); ax[0].set_title('Binary class
    distribution')
df.category.value_counts().plot.bar(ax=ax[1],color='#2E5597',rot=0); ax[1].set_title('Attack-category distribution');
    ax[1].set_yscale('log')
for a in ax: a.set_ylabel('Records')
plt.tight_layout(); plt.savefig('t1_dist.png',dpi=130); plt.close()

# label encoding
X = df.drop(columns=['attack','binary','category']).copy()
for c in ['protocol_type','service','flag']:
    X[c] = LabelEncoder().fit_transform(X[c])
le = LabelEncoder(); y = le.fit_transform(df.binary)
print('\nTarget encoding:', dict(zip(le.classes_, le.transform(le.classes_))))
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.20,random_state=42,stratify=y)
print('Training rows:',len(X_train),'| Testing rows:',len(X_test))
print('Train class share (Attack=0/Normal=1):', np.bincount(y_train)/len(y_train))

res=[]; cms={}
def evaluate(name, model, Xtr, ytr, Xte):
    t=time.time(); model.fit(Xtr,ytr); tt=time.time()-t
    p=model.predict(Xte)
    res.append(dict(Model=name, Accuracy=accuracy_score(y_test,p), Precision=precision_score(y_test,p,pos_label=0),
        Recall=recall_score(y_test,p,pos_label=0), F1=f1_score(y_test,p,pos_label=0), TrainSec=tt))
    cms[name]=confusion_matrix(y_test,p)
dt = DecisionTreeClassifier(random_state=42); evaluate('Decision Tree',dt,X_train,y_train,X_test)
rf = RandomForestClassifier(n_estimators=100,random_state=42,n_jobs=-1); evaluate('Random
    Forest',rf,X_train,y_train,X_test)
scaler=StandardScaler().fit(X_train)
Xs_tr, Xs_te = scaler.transform(X_train), scaler.transform(X_test)
rng=np.random.RandomState(42); idx=rng.choice(len(Xs_tr),20000,replace=False)
svm = SVC(kernel='rbf',C=1.0,gamma='scale'); evaluate('SVM (RBF, 20k sample)',svm,Xs_tr[idx],y_train[idx],Xs_te)
r=pd.DataFrame(res);
print('\nPerformance (Attack = positive class):'); print(r.round(4).to_string(index=False))
for n,c in cms.items(): print('\nConfusion matrix',n,'[rows actual Attack,Normal]:\n',c)
fig,ax=plt.subplots(1,3,figsize=(12,3.8))
for a,(n,c) in zip(ax,cms.items()):
```

```
    ConfusionMatrixDisplay(c,display_labels=['Attack','Normal']).plot(ax=a,colorbar=False,cmap='Blues');
    a.set_title(n,fontsize=10)
plt.tight_layout(); plt.savefig('t1_cm.png',dpi=130); plt.close()
imp=pd.Series(rf.feature_importances_,index=X.columns).sort_values(ascending=False).head(10)
print('\nTop-10 Random Forest features:'); print(imp.round(4).to_string())
plt.figure(figsize=(7,3.8)); imp[::-1].plot.barh(color='#2E5597'); plt.title('Random Forest - top 10 features');
    plt.tight_layout(); plt.savefig('t1_imp.png',dpi=130); plt.close()
```

5. Output

```
Shape: (125973, 42)
Missing values: 0 | Duplicate rows: 0
Categorical features: ['protocol_type', 'service', 'flag']
Numeric features: 38 (+3 categorical, +1 label)
int64      23
float64    15
str         4

Protocol types:
protocol_type
tcp     102689
udp      14993
icmp      8291

Unique services: 70 | unique flags: 11

Attack-name distribution (top 10):
attack
normal         67343
neptune        41214
satan           3633
ipsweep         3599
portsweep       2931
smurf           2646
nmap            1493
back             956
teardrop         892
warezclient      890

Attack-category distribution:
category
Normal    67343
DoS       45927
Probe     11656
R2L         995
U2R          52

Binary class distribution:
binary
Normal    67343
Attack    58630
binary
Normal    53.46
Attack    46.54

Target encoding: {'Attack': np.int64(0), 'Normal': np.int64(1)}
Training rows: 100778 | Testing rows: 25195
Train class share (Attack=0/Normal=1): [0.46541904 0.53458096]

Performance (Attack = positive class):
                Model  Accuracy  Precision  Recall     F1  TrainSec
        Decision Tree    0.9980     0.9978  0.9979 0.9978    0.6967
        Random Forest    0.9992     0.9995  0.9989 0.9992    5.9771
SVM (RBF, 20k sample)    0.9900     0.9873  0.9912 0.9893    1.2359

Confusion matrix Decision Tree [rows actual Attack,Normal]:
 [[11701    25]
 [   26 13443]]

Confusion matrix Random Forest [rows actual Attack,Normal]:
 [[11713    13]
 [    6 13463]]

Confusion matrix SVM (RBF, 20k sample) [rows actual Attack,Normal]:
 [[11623   103]
 [  149 13320]]

Top-10 Random Forest features:
src_bytes                0.1978
dst_bytes                0.1026
same_srv_rate            0.0773
dst_host_srv_count       0.0753
dst_host_same_srv_rate   0.0686
flag                     0.0686
```

```
logged_in                0.0424
serror_rate              0.0373
srv_serror_rate          0.0349
protocol_type            0.0324
```

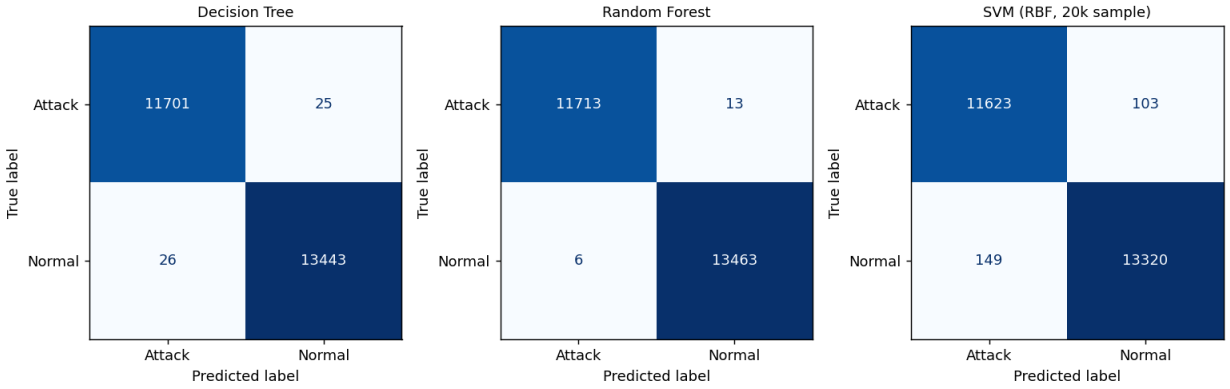


Fig 2: Confusion matrices on the 25,195 test records (rows = actual, columns = predicted).

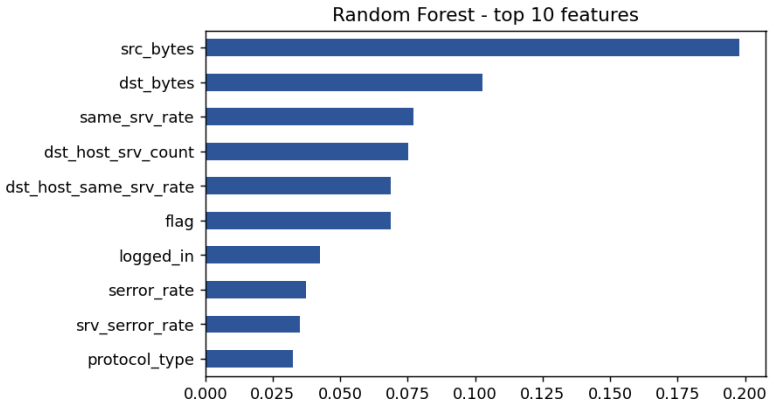


Fig 3: Top-10 Random Forest feature importances.

Result summary

Model	Accuracy	Precision	Recall	F1	Missed attacks (FN)	False alarms (FP)
Decision Tree	99.80%	99.78%	99.79%	99.78%	25	26
Random Forest	**99.92%**	99.95%	99.89%	**99.92%**	**13**	**6**
SVM (RBF, 20k sample)	99.00%	98.73%	99.12%	98.93%	103	149

6. Explanation

Working. The CSV has no header, so column names are supplied from the NSL-KDD documentation. The label column is converted into a Normal/Attack target. Categorical columns are label-encoded because scikit-learn models need numbers. The data is split 80/20 with stratification so both subsets keep the 53/47 class ratio. Each classifier is trained on the training part only and scored on the unseen test part.

Decision Tree learns if-then rules on feature thresholds (e.g. a very high serror_rate with many connections to the same service indicates a SYN-flood DoS attack). **Random Forest** trains 100 trees on bootstrap samples and takes a majority vote, which reduces the variance of a single tree. **SVM** finds the maximum-margin boundary in an RBF feature space and needs scaled inputs.

Results. Random Forest was the best model: 99.92% accuracy with only 13 missed attacks and 6 false alarms out of 25,195 records. The Decision Tree was close behind (99.80%). The SVM was lowest (99.00%) - it saw only 20,000 training rows and it treats label-encoded categories as ordered numbers, which they are not. In intrusion detection a missed attack (false negative) is more serious than a false alarm, so recall for the Attack class matters most; Random Forest also has the best recall (99.89%).

Feature importance. src_bytes and dst_bytes are the two strongest features, followed by same_srv_rate, dst_host_srv_count, flag and logged_in, which means traffic volume and connection-state behaviour separate attacks from normal traffic.

7. Limitations

- The data was split randomly from KDDTrain+, which is why scores are near 100%. On the separate official test file (KDDTest+), which contains attack types not seen in training, performance is usually noticeably lower, so these numbers are optimistic for real deployment.
- Label encoding of nominal features is acceptable for trees but not ideal for SVM; one-hot encoding would be better.
- The dataset is old (derived from 1998 traffic); R2L and U2R attacks are very rare (995 and 52 records), so a binary result hides weak detection of these classes.

8. Conclusion

All three supervised classifiers detect intrusions in NSL-KDD with over 99% accuracy. Random Forest gave the best and most balanced result (accuracy 99.92%, F1 99.92%), and is the recommended model of the three.